

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of MONDAY 5 SEPTEMBER 2022 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

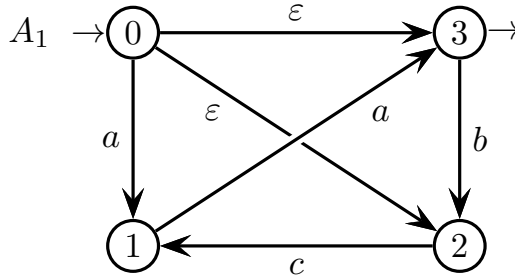
TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the following nondeterministic finite-state automaton A_1 with spontaneous transitions (ε -transitions), over the three letter alphabet $\Sigma = \{ a, b, c \}$, with initial state 0 and final state 3:

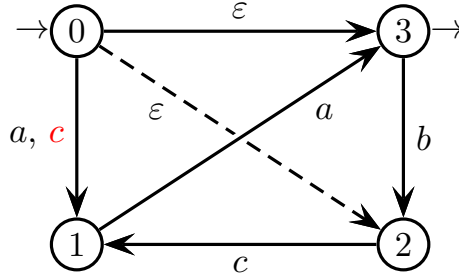


Answer the following questions:

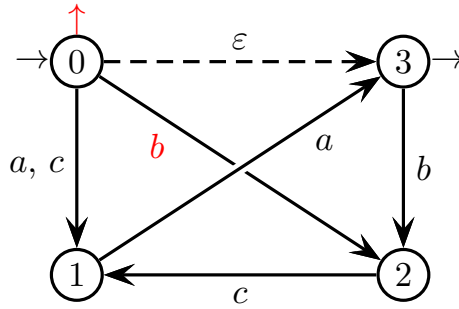
- (a) Write all the strings of language $L(A_1)$ with a length less than or equal to 3.
 - (b) Cut the spontaneous transitions of automaton A_1 and find an equivalent automaton A_2 , possibly still nondeterministic, without spontaneous transitions. Specify whether it is nondeterministic or not, and explain why.
 - (c) Verify (partially) the correctness of A_2 by testing the valid strings of point (a): list their accepting paths in A_2 .
 - (d) Apply the Berry-Sethi method (BS) to automaton A_1 and find an equivalent deterministic automaton A_3 . If necessary, minimize automaton A_3 .
 - (e) (optional) Apply the Brzozowsky-McCluskey (node elimination) method (BMC) to automaton A_1 and find an equivalent regular expression R .
-

Solution

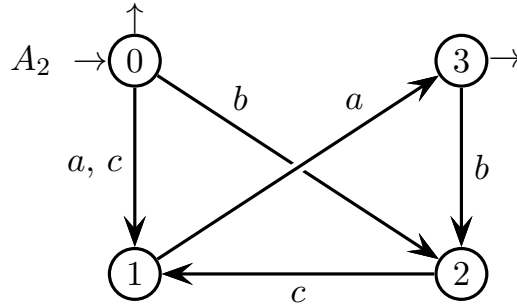
- (a) Strings $x \in L(A)$ with $|x| \leq 3$: ε , $a a$, $c a$ and $b c a$.
 (b) Cut transition $0 \xrightarrow{\varepsilon} 2$ by back-propagation:



Cut transition $0 \xrightarrow{\varepsilon} 3$ by back-propagation:



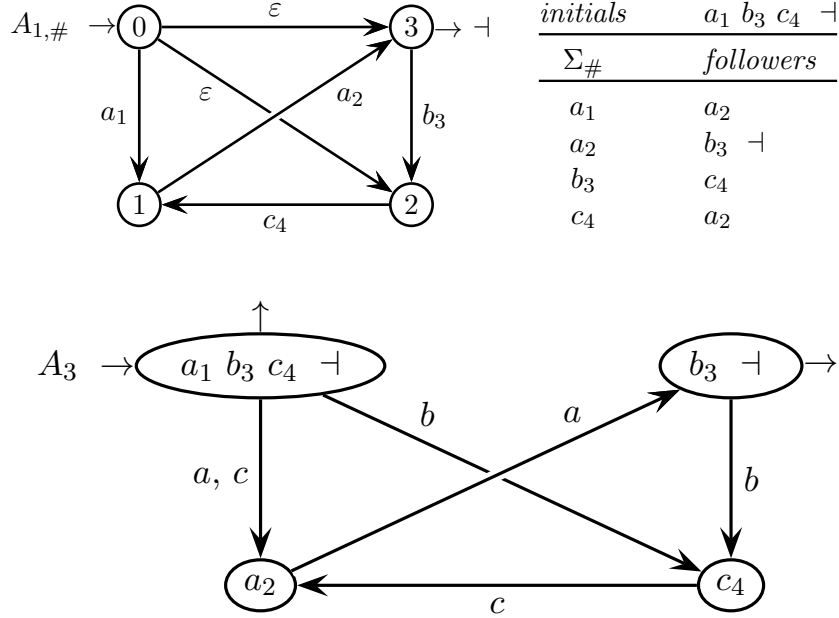
Here is the resulting automaton A_2 without spontaneous transitions



Automaton A_2 is deterministic: it has only one initial state, and it does not have any spontaneous transitions or any states with multiple outgoing transitions labeled in the same way. It is also clean, as all its states are useful, i.e., reachable and definite. Furthermore, it is easily seen to be minimal: states 0 and 3 (both final) have different outgoing arcs, i.e., for labels a and c ; and states 1 and 2 (both non-final) have different outgoing arcs as well, i.e., again for labels a and c . Therefore all the states are distinguishable and the automaton is minimal.

- (c) It is easy to see that all the strings ε , $a a$, $c a$ and $b c a$ are accepted by automaton A_2 , and that none else of length ≤ 3 is valid.

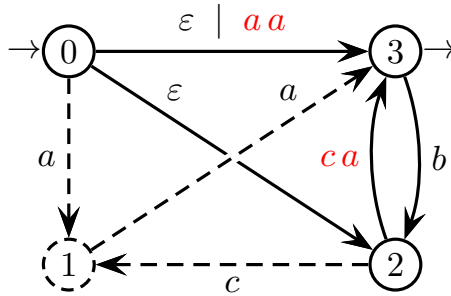
(d) Application of BS to automaton A_1 :



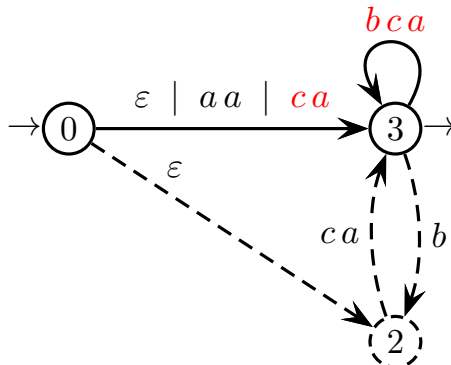
Automaton A_3 is deterministic by construction. Here it happens that it is also clean. Notice that automaton A_3 is identical to automaton A_2 (except for the state names, which is irrelevant). Since automaton A_2 is already known to be deterministic and minimal, automaton A_3 also turns out to be minimal.

(e) To apply the BMC method, automaton A_1 should be normalized. However, the initial state 0 only has outgoing arcs, so it does not need normalization, and the final state 3 has only one. Thus we can proceed informally, taking some care.

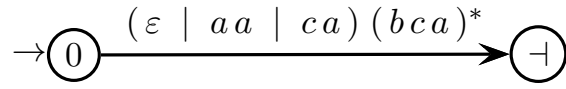
Eliminate node 1:



Eliminate node 2:



Eventually:



Therefore, the resulting regular expression R is:

$$R = (\varepsilon \mid a a \mid c a) (b c a)^*$$

or more compactly:

$$R = [(a \mid c) a] (b c a)^*$$

by factoring letter a and using the optionality operator. Expression R also passes the same (partial) correctness test as that applied to automaton A_2 .

2 Free Grammars and Pushdown Automata 20%

1. Consider the languages L_1, L_2, L_3, L_4 below, over the two-letter alphabet $\{a, b\}$:

$$L_1 = \{ a^n b^m \mid n > 0 \wedge m > 0 \wedge n \geq m \}$$

$$L_2 = \{ a^n b^m \mid n > 0 \wedge m > 0 \wedge n \leq m \}$$

$$L_3 = L_1 \cap L_2$$

$$L_4 = L_1 \cup L_2$$

Answer the following questions:

- (a) Write a grammar G_1 for language L_1 .
 - (b) Write a grammar G_3^* , of type as simple as possible, for language L_3^* (the star closure of language L_3), and say which is the smallest language family of L_3^* . Explain your answer.
 - (c) Draw a syntax tree for string $aabbaab$ according to grammar G_3^* .
 - (d) Write a grammar G_4^+ , of type as simple as possible, for language L_4^+ (the cross closure of language L_4), and say which is the smallest language family of L_4^+ . Explain your answer.
 - (e) Draw a syntax tree for string $aabbaab$ according to grammar G_4^+ .
-

Solution

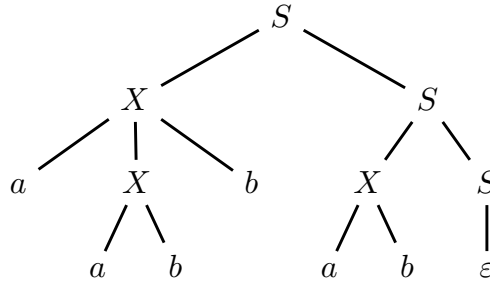
- (a) Here is a possible grammar G_1 (axiom S):

$$G_1 \begin{cases} S \rightarrow a S b \mid a T b \\ T \rightarrow a T \mid \varepsilon \end{cases}$$

- (b) The phrases of language L_3^* are lists of equal-exponent words of type $a^n b^n$, therefore the language is context-free. Here is a possible grammar G_3^* (axiom S):

$$G_3^* \begin{cases} S \rightarrow X S \mid \varepsilon \\ X \rightarrow a X b \mid a b \end{cases}$$

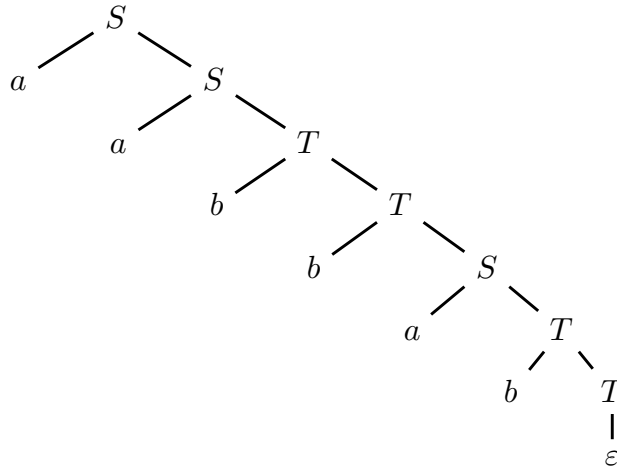
- (c) Here is the syntax tree for the string $a a b b a b$ according to grammar G_3^* :



- (d) Language $(L_4)^+ = L((a^+ b^+)^+)$ is regular and can be modeled by a right unilinear grammar. Here is a possible grammar G_4^+ (axiom S):

$$G_4^+ \begin{cases} S \rightarrow a S \mid a T \\ T \rightarrow b T \mid b S \mid \varepsilon \end{cases}$$

- (e) Here is the syntax tree for the string $a a b b a b$ according to grammar G_4^+ :



Of course, the tree is linear and basically models a list that grows on the right.

2. Consider the following sample piece of Python3 code, which defines a function:

```
def f (a, b, *args, c=1, d='s', **kwargs):  
    print (a, b, args, c, d, kwargs)
```

Answer the following questions:

- (a) Provide a grammar fragment, possibly EBNF, that supports the above kind of function definition, where the function arguments may fall into one of these four *categories*:
- zero or one or more *positional named*, e.g., *a* or *b*
 - at most one *positional anonymous*, e.g., **args*, introduced by “*”; all such arguments are collected in a single parameter (here *args*, though it can be given any valid name)
 - zero or one or more *keyword-based predefined*, e.g., *c = 1* and *d = 's'*; all such arguments are provided with a default value that can be any expression
 - at most one *keyword-based generic*, e.g., ***kwargs*, introduced by “**”; all such arguments are collected in a single parameter (here *kwargs*, though it can be given any valid name)

All the argument categories are optional, but if they are present, they must appear in the order listed above, as showcased by the legal examples below:

```
def g (*a, **kw):  
    print (a, kw)  
  
def h (a, b=1, **kw):  
    print (a, b, kw)
```

On the other hand, the following examples are illegal, as they fail to respect the argument ordering:

```
def k (b=1, a):  
    print (a, b)  
  
def l (**kw, b=1):  
    print (b, kw)
```

Use the nonterminals $\langle \text{ID} \rangle$, $\langle \text{CODE} \rangle$ and $\langle \text{EXPR} \rangle$, without expanding them, to generate the identifiers, the function body and the default values of the arguments.

- (b) Consider the type hint system of Python3, which allows a type to be attached to the function and to each argument, as showcased by the legal examples below:

```
def m (a: int, b: int, *args: int, c=1: int, d='s': str,  
      **kwargs: str) -> int:  
    print (a, b, args, c, d, kwargs)  
    return a + b + sum (args)  
  
def n (a: str, **kw: float) -> None:  
    print (a, b, kw)
```

Such type hints are always optional. Extend the grammar with the type hint system. Use the nonterminal $\langle \text{TYPE} \rangle$ for the type names, without expanding it.

- (c) (optional) Verify your solution of point (b) by drawing the syntax tree for the definition of the function *n* above.

Solution

(a) Here is a possible EBNF grammar for a function definition (axiom FUNCT):

$$\left\{ \begin{array}{l} \langle \text{FUNCT} \rangle \rightarrow \text{def } \langle \text{ID} \rangle \text{ ' (' } \overbrace{[\langle \text{ARG_LST} \rangle]}^{\text{optional args}} \text{ ' , ' ' : ' } \langle \text{CODE} \rangle \\ \hline \langle \text{ARG_LST} \rangle \rightarrow \langle \text{CAT1} \rangle \\ \langle \text{CAT1} \rangle \rightarrow \overbrace{[\langle \text{PON_LST} \rangle \text{ ' , ' }]}^{\text{optional pos. named}} \langle \text{CAT2} \rangle \mid \langle \text{PON_LST} \rangle \\ \langle \text{CAT2} \rangle \rightarrow \overbrace{[\text{ ' * ' } \langle \text{ID} \rangle \text{ ' , ' }]}^{\text{optional pos. anon.}} \langle \text{CAT3} \rangle \mid \text{ ' * ' } \langle \text{ID} \rangle \\ \langle \text{CAT3} \rangle \rightarrow \overbrace{[\langle \text{KWB_LST} \rangle \text{ ' , ' }]}^{\text{optional kw-based pred.}} \langle \text{CAT4} \rangle \mid \langle \text{KWB_LST} \rangle \\ \langle \text{CAT4} \rangle \rightarrow \text{ ' ** ' } \langle \text{ID} \rangle \quad // \text{ here kw-based gen. is mandatory} \\ \hline \langle \text{PON_LST} \rangle \rightarrow \langle \text{ID} \rangle \text{ (' , ' } \langle \text{ID} \rangle \text{) }^* \\ \langle \text{KWB_LST} \rangle \rightarrow \langle \text{DEF_ASG} \rangle \text{ (' , ' } \langle \text{DEF_ASG} \rangle \text{) }^* \\ \hline \langle \text{DEF_ASG} \rangle \rightarrow \langle \text{ID} \rangle \text{ ' = ' } \langle \text{EXPR} \rangle \end{array} \right.$$

As said, the nonterminals ID, CODE and EXPR are left unexpanded. At the grammar core, there is the rule chain CAT1/2/3/4 that generates the function arguments in the requested category order. According to such rules:

- starting from ARG_LST, the four argument categories are generated in the specified order through the chain CAT1/2/3/4, with the separator comma
- each one of the four categories is optional, independently of the presence of the other three, with an exception:
 - if the first three categories are all absent, the fourth one has to be present (in fact, if the nonterminal CAT4 were nullable, the nonterminal ARG_LST could generate an argument list with an orphan comma at the end)
 - notice however that if besides the first three categories, also the fourth one had to be absent, then the function would have no arguments at all
 - therefore, this tiny loss of symmetry is fully repaired by allowing the function to be declared without arguments from the very beginning (rule FUNCT)

Thus, the modeling of the arguments and of their ordering is reasonably correct. See also the syntax tree of the last question. Nonterminal ARG_LST is motivated for structural elegance, though it can be easily eliminated and replaced with CAT1. The rule stratification is shown to make the grammar more readable.

Furthermore, the grammar is reasonably non-ambiguous, since it uses lists constructed in a non-ambiguous way (rules PON_LST and KWB_LST), and there is not any union ambiguity (rules CAT1/2/3/4). More algorithmically, by rewriting the grammar as a machine net, it would not be difficult to prove that it is of type ELL(k)¹, for some suitable value $k \geq 1$, and consequently non-ambiguous.

¹At least, after lexical analysis, when the identifiers, which can reasonably be modeled with a single regular expression over the terminal alphabet, have become atomic objects ID.

- (b) To model the grammar extension that includes the (optional) type hint system, the axiomatic rule must be modified as follows:

$$\langle \text{FUNCT} \rangle \rightarrow \text{def } \langle \text{ID} \rangle ' (' [\langle \text{ARG_LST} \rangle] ') ' [' \rightarrow ' \langle \text{TYPE} \rangle] ' : ' \langle \text{CODE} \rangle$$

Furthermore, the instances of nonterminal ID in the rules CAT2/4 and PON_LST must be replaced by a new nonterminal TYPED_ID, a new rule must be added to expand TYPED_ID, and rule DEF_ASG must be modified, as follows:

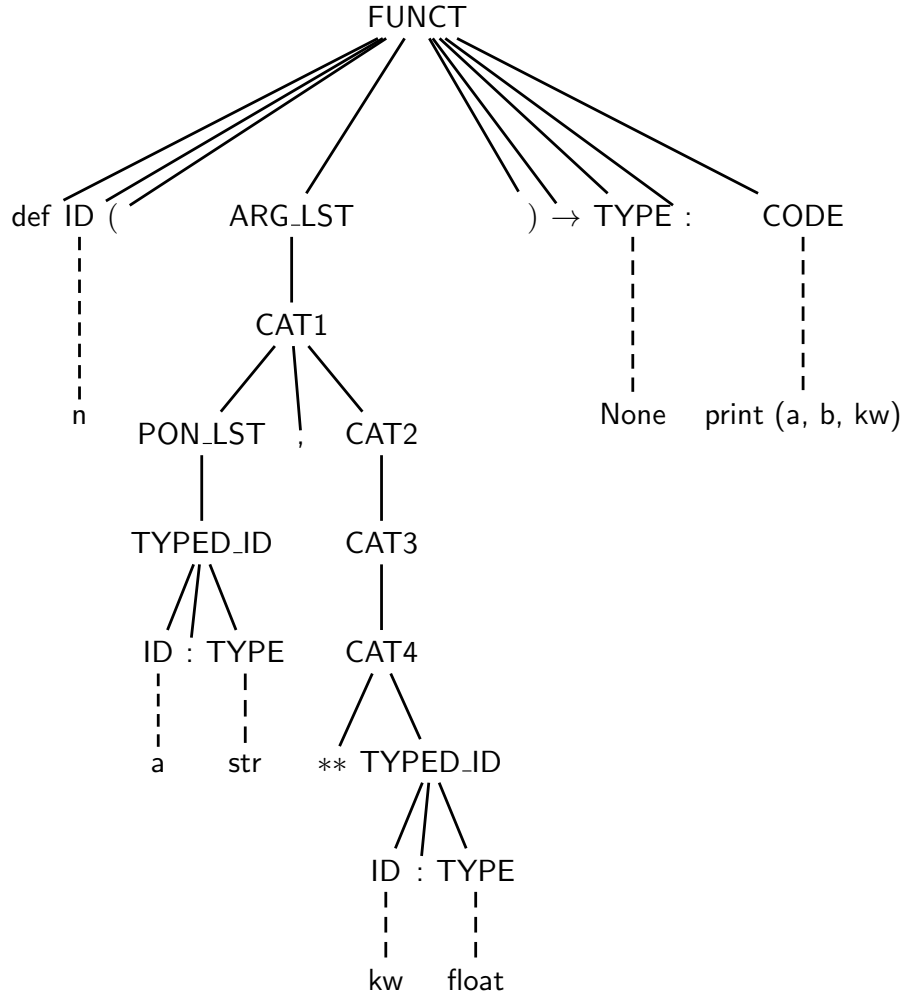
$$\langle \text{TYPED_ID} \rangle \rightarrow \langle \text{ID} \rangle [' : ' \langle \text{TYPE} \rangle]$$

$$\langle \text{DEF_ASG} \rangle \rightarrow \langle \text{ID} \rangle ' = ' \langle \text{EXPR} \rangle [' : ' \langle \text{TYPE} \rangle]$$

The nonterminal TYPE is left unexpanded. The grammar is still unambiguous.

- (c) Here is the syntax tree of function n, for the extended grammar (point (b)):

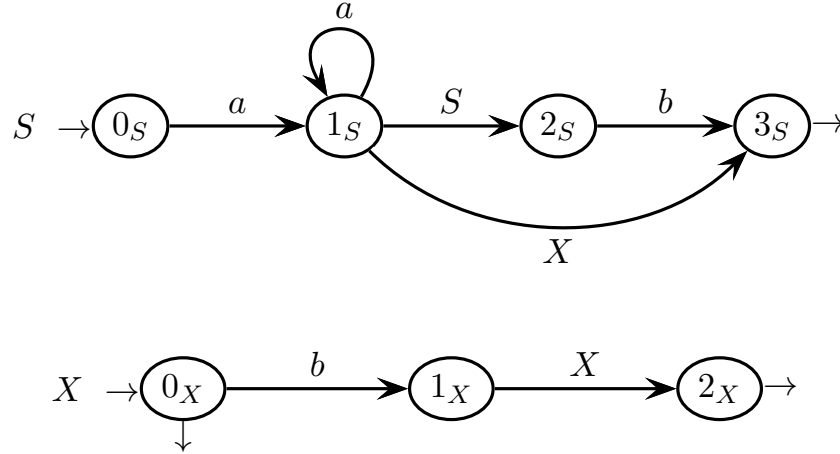
```
def n (a: str, **kw: float) -> None: print (a, b, kw)
```



The subtrees that are not modeled in detail (ID, TYPE and CODE) are condensed in one dashed edge. Notice in particular the rule chain CAT1/2/3/4: for rule CAT1 the optional part is taken, while for both rules CAT2 and CAT3 it is not (read the previous explanation at point (a) for more clarification).

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented as a machine net, over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ and the two-symbol nonterminal alphabet $V_N = \{ S, X \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

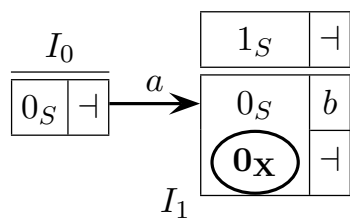
- (a) Draw the complete pilot of grammar G and show that grammar G is not of type ELR (1), by listing all the pilot conflicts.
- (b) Find all the guide sets on the call arcs and exit arrows of the machine net, and show that grammar G is not of type ELL (1), by listing all the guide set conflicts.
- (c) Is grammar G ambiguous or not ? Explain your answer.
- (d) Simulate the ELR recognition of the valid string below (this string does not meet any conflict):

$$a a a \in L(G)$$

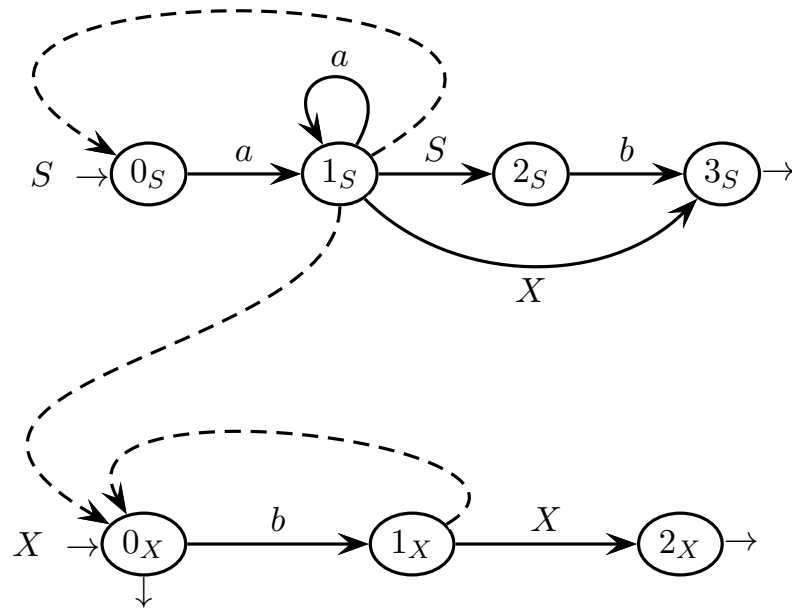
Also show the construction of the syntax tree of the string.

- (e) (optional) What is the smallest family to which language $L(G)$ belongs ? Explain your answer.

pilot to be completed with all m-states and arcs – question (a)



machine net to be completed with the guide sets – question (b)



space for answering question (c)

simulation table for answering question (d)

ELR simulation table – string $a a a$ – number of rows not significant

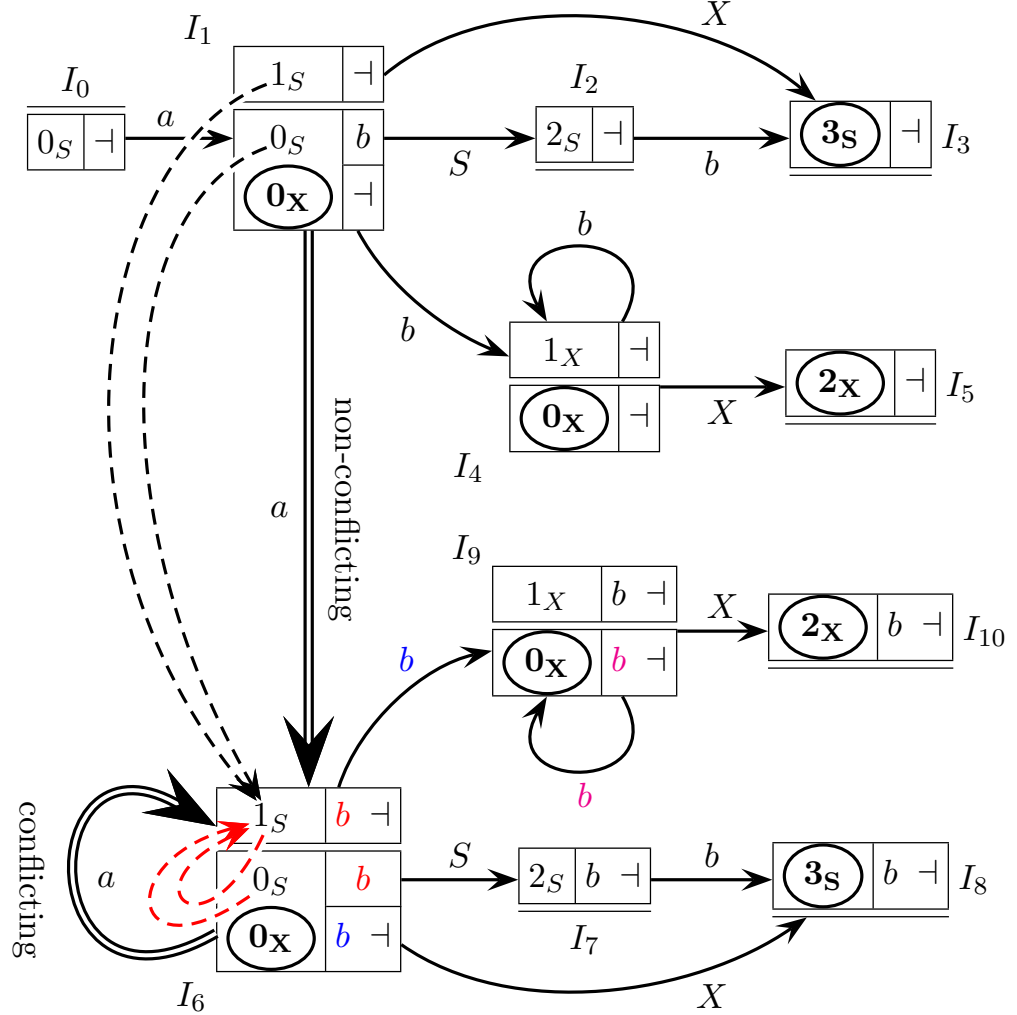
<i>stack of m-states and grammar symbols</i>	<i>input left to scan</i>	<i>move executed</i>
I_0	$a a a \dashv$	initial configuration
$I_0 a I_1$	$a a \dashv$	terminal shift: $I_0 \xrightarrow{a} I_1$

show here the construction of the tree – question (d)

space for answering question (e)

Solution

(a) Here is the complete pilot, with eleven m-states (and two double transitions):



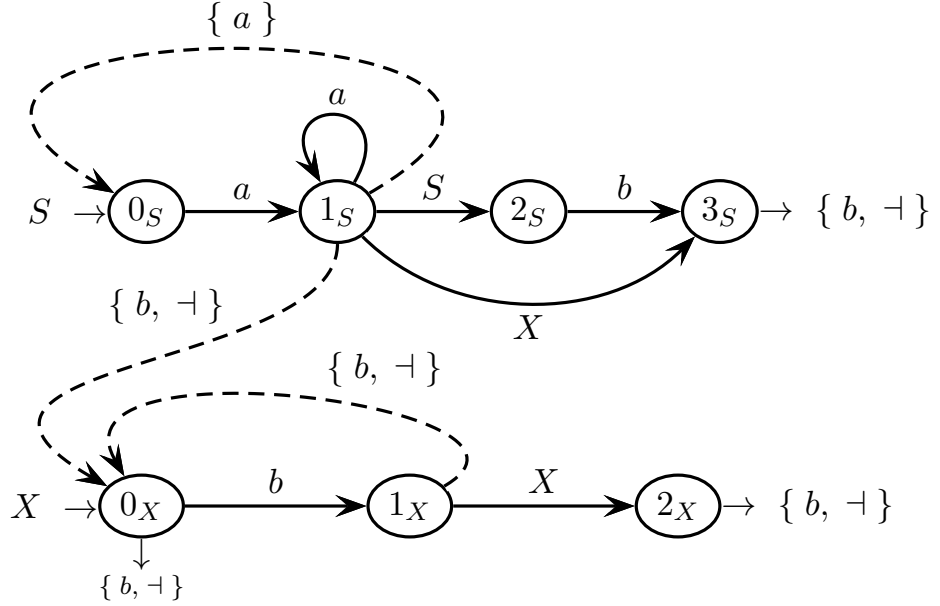
Here are all the conflicts in the pilot (convergent transitions are doublelined):

- There is a shift-reduce conflict on terminal b in each of the m-states I_6 and I_9 .
- There are no reduce-reduce conflicts anywhere, as no m-state contains two or more final net states.
- There is a convergence conflict: first, the pilot transition $I_6 \xrightarrow{a} I_6$ (a pilot self-loop) is double, as it consists of the two net transitions $0_S \xrightarrow{a} 1_S$ and $1_S \xrightarrow{a} 1_S$ (the latter is a net self-loop); second, such net transitions share their destination state 1_S , so that the pilot transition is convergent; and third, they share terminal b in their look-ahead sets, so that convergence is conflicting.²

There are not any other conflicts. Clearly grammar G is not ELR(1).

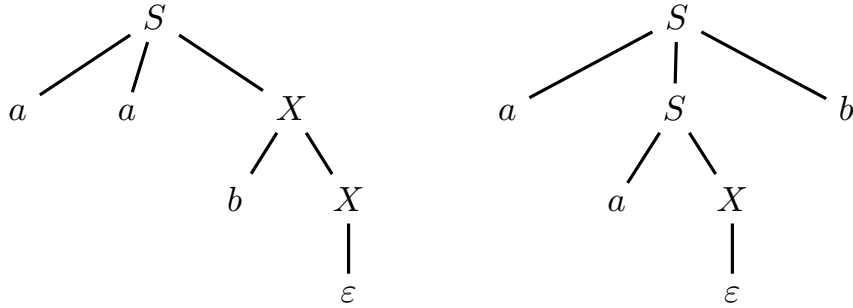
²Notice that the pilot transition $I_1 \xrightarrow{a} I_6$ is double and convergent as well. However, in this case the two component net transitions have disjoint look-ahead sets ($\neg$ and b), so that convergence is not conflicting.

(b) Here are all the guide sets on the call arcs and exit arrows (for $k = 1$):



There are one guide set conflict on state 1_S for letter a , and one on state 0_X for letter b . Clearly grammar G is not ELL(1). It would be easy to see that it is not ELL(k) either, for any $k > 1$ (as it is ambiguous, see point (c) below).

(c) Grammar G is ambiguous. For instance, string aab is ambiguous with two trees:



Actually, grammar G is *heavily* ambiguous. In fact, all the valid strings that contain one or more letters b (except for the one string ab) are ambiguous, since there are two distinct generation mechanisms for such letters, i.e., through recurrence on nonterminal S or through recurrence on nonterminal X .

More in detail, clearly the number of trees of a string that contains letters b grows with the number of such letters, because there are more and more choices on how many letters b to generate through nonterminal S rather than nonterminal X . Anyway, the number of trees remains finite for every string, as grammar G does not have circular nonterminals and its only null rule, i.e., $X \rightarrow \varepsilon$, can be applied only once (at the derivation end). Therefore grammar G is unlimitedly ambiguous, though it is not infinitely ambiguous³.

³Remember that for a string to have infinitely many trees, those have to contain linear branches of increasing length (so the derivation should use one or more circular nonterminals) or to contain increasingly many empty leaf nodes (so the derivation should use one or more null rules for as many times).

Clearly too, since nonterminal X is nullable, all the strings of type a^+ belong to language $L(G)$, and none of those is ambiguous. Actually, those are the only non-ambiguous strings (see also point (e)), plus string ab . The latter is a limit case: since there is only one letter a , the letter b can only be generated by X .

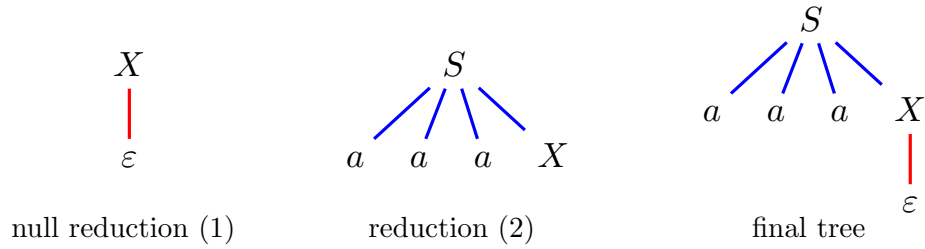
- (d) It does not take long to simulate string aaa , as there are only a (null) reduction for nonterminal X and the final reduction to the axiom S :

ELR simulation table – string aaa

<i>stack of m-states and grammar symbols</i>	<i>input left to scan</i>	<i>move executed</i>
I_0	$aaa \dashv$	initial configuration
$I_0 a I_1$	$aa \dashv$	terminal shift: $I_0 \xrightarrow{a} I_1$
$I_0 a I_1 a I_6$	$a \dashv$	terminal shift: $I_1 \xrightarrow{a} I_6$
$I_0 a I_1 a I_6 a I_6$	$\dashv$	terminal shift: $I_6 \xrightarrow{a} I_6$
$I_0 a I_1 a I_6 a I_6$	$\dashv$	null reduction (1): $\varepsilon \rightsquigarrow X$
$I_0 a I_1 a I_6 a I_6 X I_8$	$\dashv$	nonterminal shift: $I_6 \xrightarrow{X} I_8$
I_0	$\dashv$	reduction (2): $aaaX \rightsquigarrow S$
initial m-state	empty tape	last move: reduction to axiom

acceptance condition verified – accept

Here is the tree construction, with the elementary trees of each reduction and the final tree of the whole string:



Notice that the two terminal shift moves $I_1 \xrightarrow{a} I_6$ and $I_6 \xrightarrow{a} I_6$ involve the two (double) convergent transitions in the pilot (see point (a)). The former transition is non-conflicting, so that it does not break determinism. The latter one is conflicting, but on reading the third (and last) letter a of string aaa the look-ahead in the input is the terminator $\dashv$ (since the string has ended), which is not shared between the two net transitions composing the pilot transition; therefore, determinism is not broken in this case either. It is a limit case however, as by appending a letter b to aaa , the look-ahead in the input becomes letter b , which is shared, and this breaks determinism and even makes string $aaab$ ambiguous.

- (e) The smallest family of language $L(G)$ turns out to be the regular one. Since machine M_X maps to the right-linear rule:

$$X \rightarrow bX \mid \varepsilon$$

we have:

$$L(X) = b^*$$

Since the axiomatic machine M_S maps to the self-embedding rule:

$$S \rightarrow a a^* (Sb \mid X) = a^+ Sb \mid a^+ X$$

we have (consider the well-known fact that the rule model $Z \rightarrow a Z b \mid c$ generates language $L(Z) = a^h c b^h$ for all $h \geq 0$):

$$L(S) = (a^+)^h L(a^+ X) b^h = (a^+)^{h+1} L(X) b^h$$

for all $h \geq 0$ ⁴. By substitution of $L(X)$ into $L(S)$ we have:

$$L(S) = (a^+)^{h+1} b^* b^h$$

for all $h \geq 0$. Since $h \geq 0$, we then have $(a^+)^{h+1} = a^+$ and $b^* b^h = b^*$. Thus neither the a -substring nor the b -substring depend on the exponent h , and consequently their lengths are unrelated to each other. Therefore by substitution the language generated by grammar G is as follows:

$$L(G) = a^+ b^*$$

which is regular and clearly not finite. Therefore the smallest family language $L(G)$ belongs to is the regular one. We can also observe that the language is not inherently ambiguous (as it is regular). Actually, it is fully deterministic.

⁴For simplicity of notation, we have written $L(X) = b^*$ instead of $L(X) = L(b^*)$, $L(Z) = a^h c b^h$ instead of $L(Z) = \{a^h c b^h \mid h \geq 0\}$, and $L(S) = (a^+)^{h+1} L(X) b^h$ instead of $L(S) = \bigcup_{h \geq 0} L((a^+)^{h+1} L(X) b^h)$, and similarly in the lines below.

4 Language Translation and Semantic Analysis 20%

1. The source BNF grammar G^s below, with terminal alphabet $\Sigma = \{ b, e, i, s \}$, non-terminal alphabet $V = \{ B, E, L \}$ and axiom B , generates an abstract representation of an instruction block. A block B has an opening delimiter b and a closing delimiter e , and it includes a list L of instructions E . An instruction E may be simple, denoted by terminal i , or may be structured, i.e., a nested block. Between any two consecutive instructions in a list inside a block, there is a *separator* denoted s . Here is grammar G^s :

$$G^s \begin{cases} B \rightarrow b L e \\ L \rightarrow E s L \mid E \\ E \rightarrow i \mid B \end{cases}$$

The phrase below is an instance of an instruction block:

$$\underline{b \ i \ s \ \underline{b \ i \ e} \ s \ i \ e}$$

This phrase has two nested blocks. The outer block contains three instructions, one of which (the middle one) is a block that in turn includes one simple instruction.

We want to define a translation τ_1 so that in the target sentence:

- (a) the block delimiters are not b and e , but p and c , respectively
- (b) instead of having a separator s between consecutive list elements, each element is followed by a *terminator* t

Therefore, for instance:

$$\tau_1(b \ i \ s \ b \ i \ e \ s \ i \ e) = p \ i \ t \ p \ i \ t \ c \ t \ i \ t \ c$$

Answer the following questions:

- (a) Write a target grammar G_1^t that defines the translation τ_1 , with G^s as its source grammar.
- (b) Draw the source and target trees for the translation example below:

$$\tau_1(b \ i \ s \ b \ i \ e \ s \ i \ e) = p \ i \ t \ p \ i \ t \ c \ t \ i \ t \ c$$

- (c) Next consider a slightly different translation τ_2 , where the terminator t follows all the list elements of simple type, i.e., i , but not those of block type, i.e., “ $p \dots c$ ”. For instance:

$$\tau_2(b \ i \ s \ b \ i \ e \ s \ i \ e) = p \ i \ t \ p \ i \ t \ c \ i \ t \ c$$

Write a target grammar G_2^t , still based on G^s , that defines translation τ_2 .

Solution

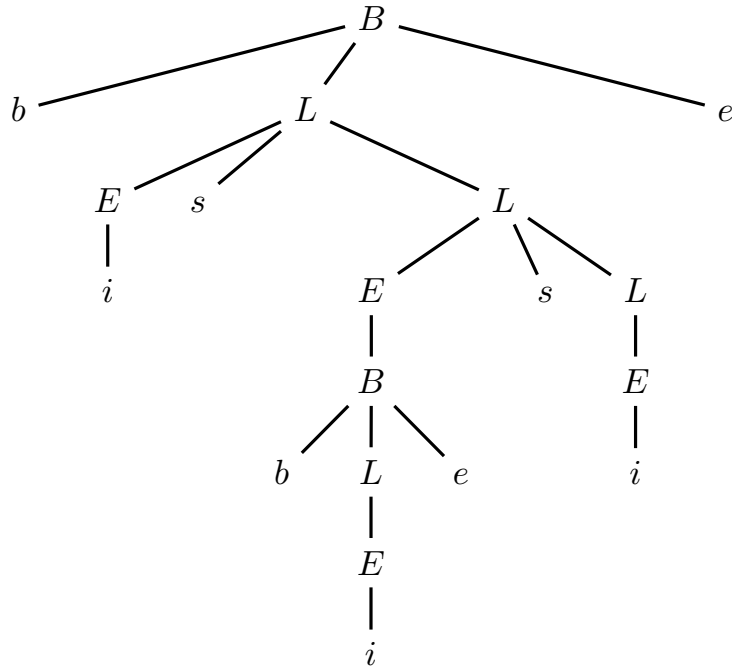
(a) Here is a possible target grammar G_1^t :

$$G_1^t \left\{ \begin{array}{l} B \rightarrow p L c \\ L \rightarrow E L \mid E \\ E \rightarrow i t \mid B t \end{array} \right.$$

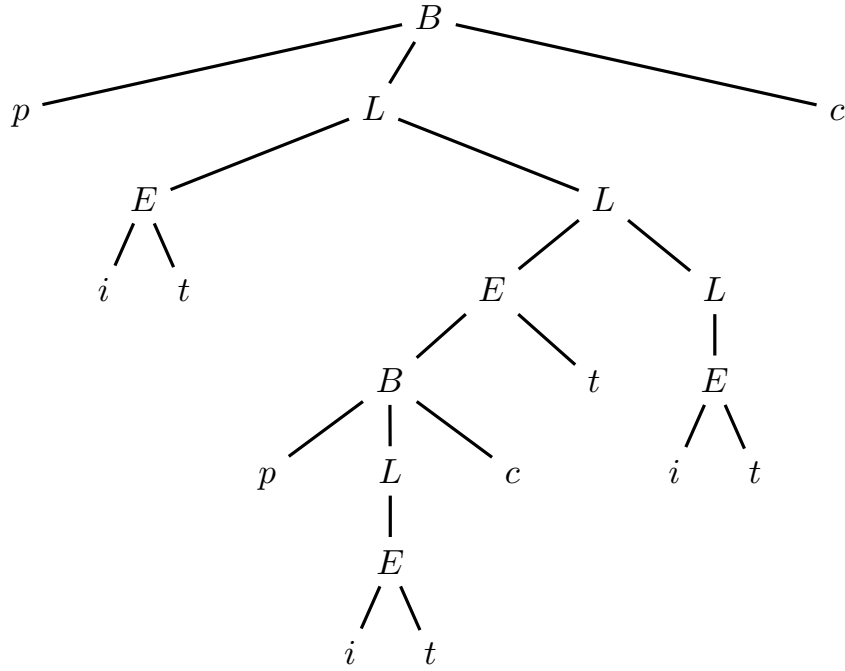
The variant $\overline{G}_1^t$ below is acceptable as well:

$$\overline{G}_1^t \left\{ \begin{array}{l} B \rightarrow p L c \\ L \rightarrow E t L \mid E t \\ E \rightarrow i \mid B \end{array} \right.$$

(b) Here is the source tree for string $b i s b i e s i e$:



and below the corresponding target tree, for string $p i t p i t c t i t c$:



The internal structure of the two trees is the same, of course. Only the leaf nodes are different.

(c) Here is a possible target grammar G_2^t :

$$G_2^t \left\{ \begin{array}{l} B \rightarrow p L c \\ L \rightarrow E L \mid E \\ E \rightarrow i t \mid B \end{array} \right.$$

The trees (source and target) are a simple variant of the two above, and are left to the reader.

2. Consider a BNF grammar that defines a data type, as follows (axiom `Type`):

$$\left\{ \begin{array}{l} \langle \text{Type} \rangle \rightarrow \text{'\{'} \langle \text{Recltems} \rangle \text{'\}'} \text{id} \\ \langle \text{Type} \rangle \rightarrow \langle \text{Array} \rangle \\ \langle \text{Type} \rangle \rightarrow \langle \text{BaseType} \rangle \\ \langle \text{Recltems} \rangle \rightarrow \langle \text{Type} \rangle \text{'\;'} \langle \text{Recltems} \rangle \\ \langle \text{Recltems} \rangle \rightarrow \langle \text{Type} \rangle \text{'\;'} \\ \langle \text{Array} \rangle \rightarrow \langle \text{Type} \rangle \text{'['} \text{num} \text{'\}]'} \\ \langle \text{BaseType} \rangle \rightarrow \text{int id} \\ \langle \text{BaseType} \rangle \rightarrow \text{char id} \end{array} \right.$$

We want to define an attribute grammar that computes the size (measured in bytes) of each element of the data type. To this end, we introduce a synthesized (left) integer attribute *size* associated with all the nonterminals: `Type`, `Recltems`, `Array` and `BaseType`. Assume the sizes of the base types `char` and `int` to be one byte and four bytes, respectively, and the numerical value of a terminal `num` to be denoted as n .

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write the semantic functions that compute the attribute *size* for all the nonterminals of the grammar.
- (b) Compute the attribute *size* for the syntax tree of the data structure below:

```
{
    char alpha;
    int vect [6];
} data
```

- (c) (optional) We want to extend the attribute grammar and add to it the computation of the memory offset (measured in bytes) of each element of the data structure. To this end, we introduce an inherited (right) integer attribute *offset* associated with the nonterminals `Type`, `Recltems` and `Array`. Subsequent elements of the data type are allocated consecutively in the memory. Add to the previous grammar the appropriate semantic functions for computing the attribute *offset*.

table to be filled in for question (a)

#	<i>syntax</i>	<i>semantic functions</i>
1:	$\langle \text{Type} \rangle_0 \rightarrow \text{'{' } \langle \text{RecItems} \rangle_1 \text{'}' id}$	
2:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{Array} \rangle_1$	
3:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{BaseType} \rangle_1$	
4:	$\langle \text{RecItems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' ; ' } \langle \text{RecItems} \rangle_2$	
5:	$\langle \text{RecItems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' ; '}$	
6:	$\langle \text{Array} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' [' num ']'}$	
7:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{int id}$	
8:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{char id}$	

tree to be decorated for question (b)

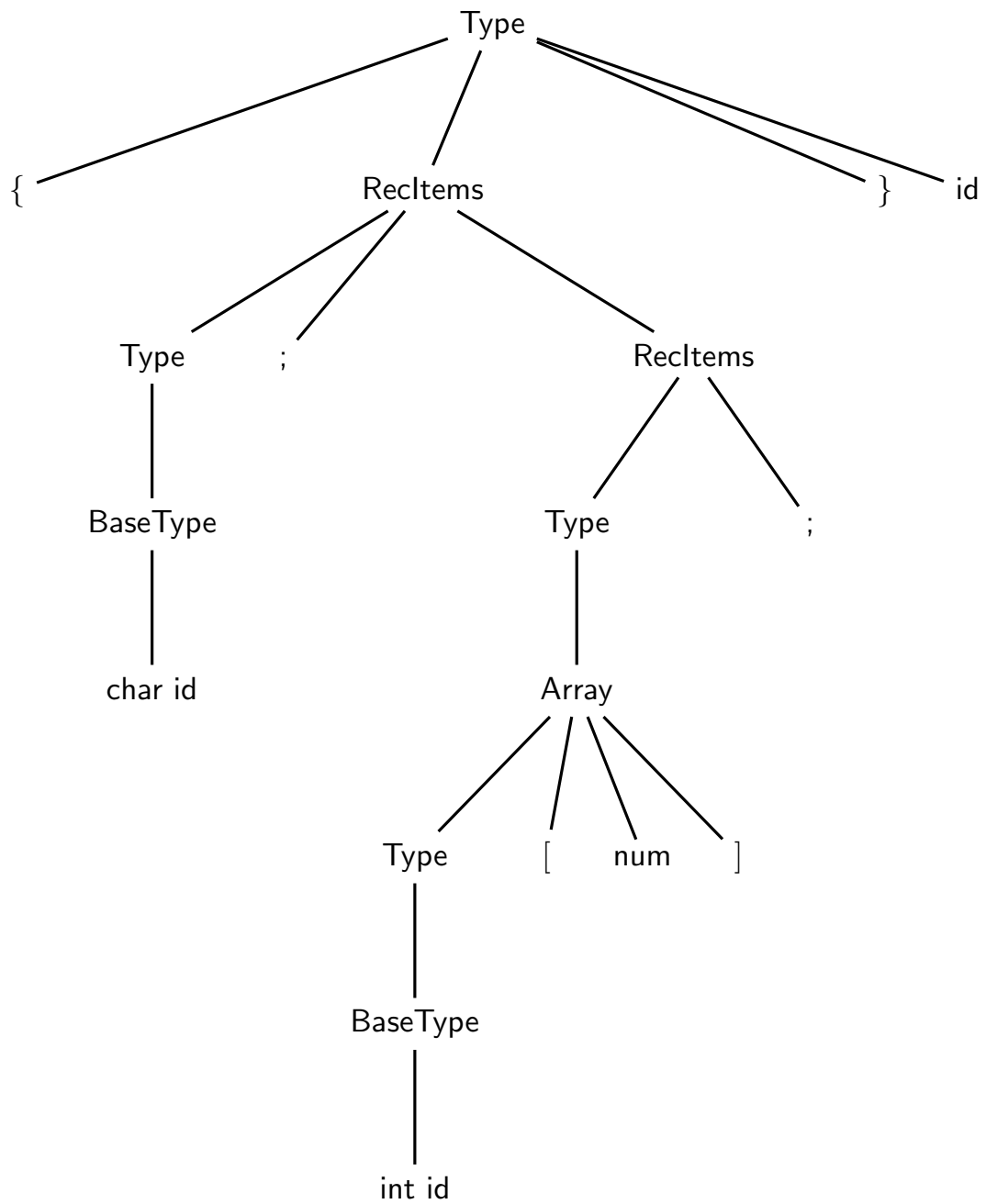


table to be filled in for question (c)

#	<i>syntax</i>	<i>semantic functions</i>
1:	$\langle \text{Type} \rangle_0 \rightarrow \text{'{' } \langle \text{RecItems} \rangle_1 \text{'}' id}$	
2:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{Array} \rangle_1$	
3:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{BaseType} \rangle_1$	
4:	$\langle \text{RecItems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' ; ' } \langle \text{RecItems} \rangle_2$	
5:	$\langle \text{RecItems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' ; '}$	
6:	$\langle \text{Array} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{' [' num ']'}$	
7:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{int id}$	
8:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{char id}$	

Solution

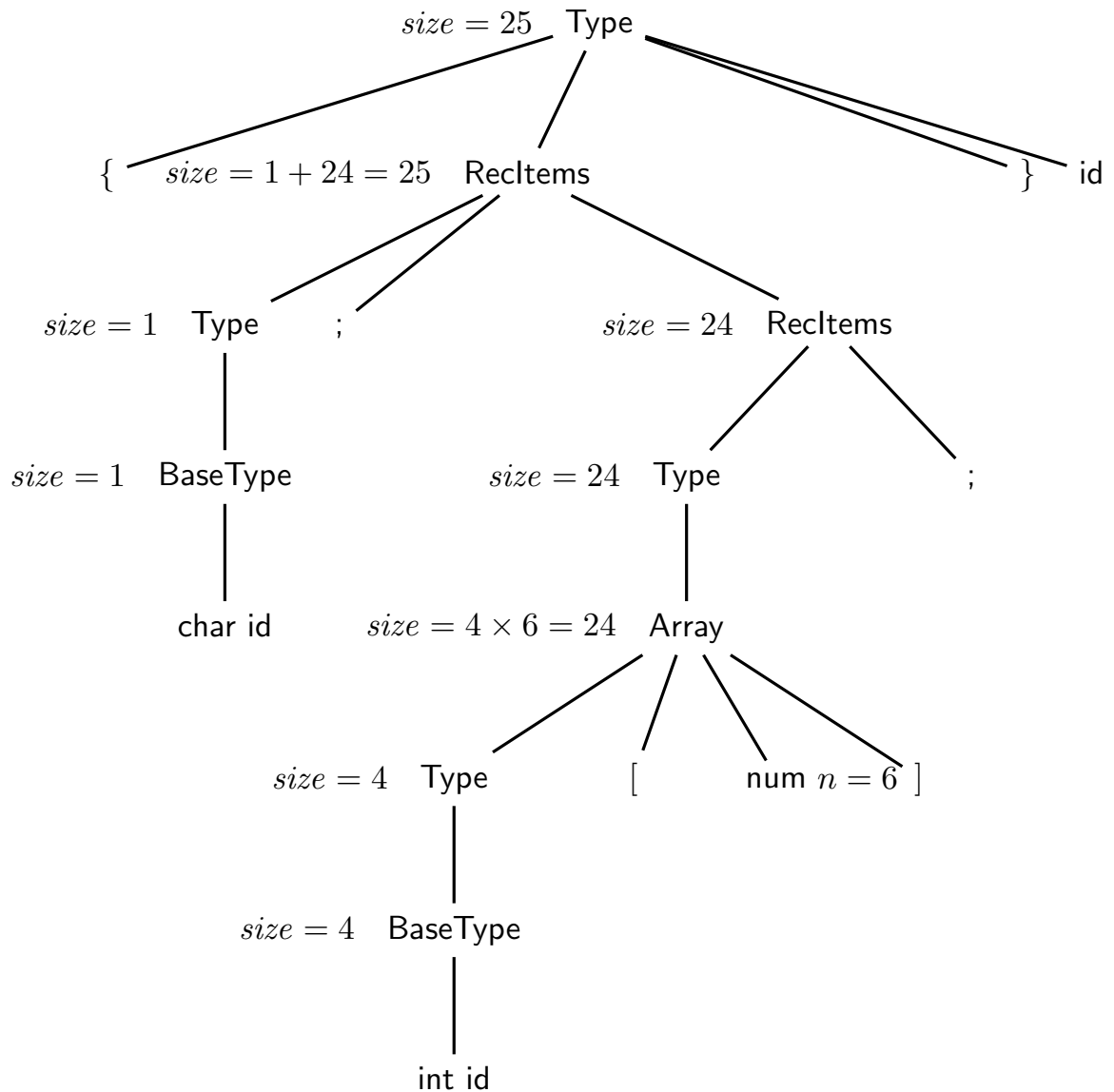
(a) Here is the attribute grammar for computing the size:

#	<i>syntax</i>	<i>semantic functions</i>
1:	$\langle \text{Type} \rangle_0 \rightarrow \text{'{' } \langle \text{Recltems} \rangle_1 \text{'}' id}$	$size_0 := size_1$
2:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{Array} \rangle_1$	$size_0 := size_1$
3:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{BaseType} \rangle_1$	$size_0 := size_1$
4:	$\langle \text{Recltems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{';' } \langle \text{Recltems} \rangle_2$	$size_0 := size_1 + size_2$
5:	$\langle \text{Recltems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{';'}$	$size_0 := size_1$
6:	$\langle \text{Array} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{'[' num '[']}$	$size_0 := size_1 \times n$
7:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{int id}$	$size_0 := 4$
8:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{char id}$	$size_0 := 1$

The grammar is correct: no circular dependencies. Notice that the grammar is purely synthesized. As such, it is of type one-sweep.

(b) Here are the values of attribute *size* for the tree of the data structure below:

```
{
  char alpha;
  int vect [6];
} data
```



Attribute *size* is of type left and is annotated on the left side of the inner node. Attribute *n* is associated to a terminal, here **num**, which means it is a function of the lexical token **num**. Conventionally an attribute of a terminal is considered to be of type right, and is annotated on the right side of the leaf node.

- (c) Here is the attribute grammar for computing the memory offset. The grammar reuses the attribute *size* and introduces the inherited attribute *offset*, which is associated to the nonterminals *Type*, *Recltems* and *Array* (not to *BaseType*):

#	<i>syntax</i>	<i>semantic functions</i>
1:	$\langle \text{Type} \rangle_0 \rightarrow \text{'{' } \langle \text{Recltems} \rangle_1 \text{'}' id}$	$offset_1 := offset_0$ $size_0 := size_1$
2:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{Array} \rangle_1$	$offset_1 := offset_0$ $size_0 := size_1$
3:	$\langle \text{Type} \rangle_0 \rightarrow \langle \text{BaseType} \rangle_1$	$size_0 := size_1$
4:	$\langle \text{Recltems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{';' } \langle \text{Recltems} \rangle_2$	$offset_1 := offset_0$ $offset_2 := offset_1 + size_1$ $size_0 := size_1 + size_2$
5:	$\langle \text{Recltems} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{';'}$	$offset_1 := offset_0$ $size_0 := size_1$
6:	$\langle \text{Array} \rangle_0 \rightarrow \langle \text{Type} \rangle_1 \text{'[' num ']'}$	$offset_1 := offset_0$ $size_0 := size_1 \times n$
7:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{int id}$	$size_0 := 4$
8:	$\langle \text{BaseType} \rangle_0 \rightarrow \text{char id}$	$size_0 := 1$

The grammar is correct: no circular dependencies.

Notice that for rule 4 we could equivalently write $offset_2 := offset_0 + size_1$, since immediately before we have $offset_1 := offset_0$. Such a variant is correct, yet it would cause the attribute $offset_2$ to depend also on the attributes of the parent node *Recltems*, instead of depending only on those of the brother node *Type*.

The attribute *offset* is left uninitialized in the tree root. It will be simply assigned the initial offset, whatever it is, chosen by the compiler for the data segment. For instance, supposing that the axiom were a new nonterminal *DataSeg* and that the axiomatic rule were $\langle \text{DataSeg} \rangle_0 \rightarrow \langle \text{Type} \rangle_1$, we could set $offset_1 := 0$, if the compiler allocates the data segment in the memory starting from address 0.

- (d) The attribute grammar for computing the memory offset is of type one-sweep, since the right attribute *offset* is only inherited top-down (in all the rules from 1 to 6, except for 3) and left-to-right (in the rule 4), and since it depends only on (itself and on) the purely synthesized attribute *size* (in the rule 4).